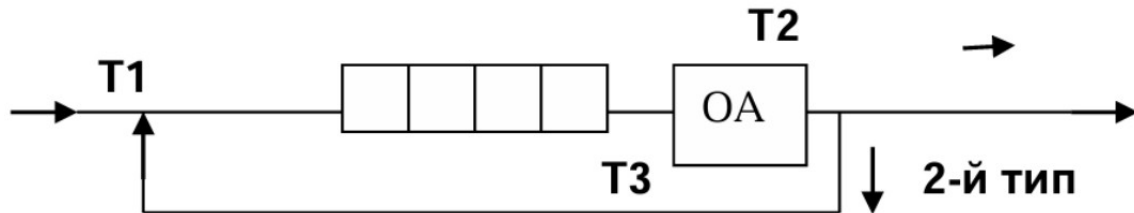


1. Описание условия задания

Цель работы:

Система массового обслуживания состоит из обслуживающего аппарата (ОА) и очереди заявок двух типов.



Заявки 1-го типа поступают в "хвост" очереди по случайному закону с интервалом времени $T1$, равномерно распределенным от 0 до 5 единиц времени (е.в.). В ОА они поступают из "головы" очереди по одной и обслуживаются также равновероятно за время $T2$ от 0 до 4 е.в., после чего покидают систему.

Единственная заявка 2-го типа постоянно обращается в системе, обслуживаясь в ОА равновероятно за время $T3$ от 0 до 4 е.в. и возвращаясь в очередь на 4-ю позицию от "головы" (или в конец очереди, если ее длина меньше 3). В начале процесса заявка 2-го типа входит в ОА, оставляя пустую очередь. (Все времена – вещественного типа) Смоделировать процесс обслуживания первых 1000 заявок 1-го типа.

Выдавать после обслуживания каждых 100 заявок 1-го типа информацию о текущей и средней длине очереди, количестве вошедших и вышедших заявок и о среднем времени пребывания заявок в очереди. В конце процесса выдать общее время моделирования, время простоя аппарата, количество вошедших в систему и вышедших из нее заявок первого типа и количество обращений заявок второго типа. По требованию пользователя выдать на экран адреса элементов очереди при удалении и добавлении элементов. Проследить, возникает ли при этом фрагментация памяти.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные:

- Элементы очереди в виде целых чисел, вводимых пользователем.
- Выбор операции через консольное меню.
- Параметры моделирования.

Выходные данные:

- Текущее состояние очереди.
- Адреса элементов для списка.
- Результаты моделирования с использованием массива или списка.
- Сравнение времени выполнения и объема памяти для двух реализаций.

Ограничения:

- Максимальный размер очереди: 1000 элементов.
- Параметры обработки заявок двух типов целые, неотрицательные, максимальное время больше минимального.

Способ обращения: консольный ввод/вывод.

Команда запуска программы: ./app.exe. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- “ERROR_MEM” – Ошибка выделения памяти.
- “ERROR_IO” – Ошибка ввода.
- “ERROR_OVERFLOW” – Попытка добавить элемент в полную очередь.
- “ERROR_DELETING” – Попытка удалить элемент из пустой очереди.

Возможные действия, реализуемой программой:

0. Выход.
1. Очередь-массив: Добавление элемента.
2. Очередь-массив: Удаление элемента.
3. Очередь-массив: Вывод очереди.
4. Очередь-массив: Моделирование очереди.
5. Очередь-список: Добавление элемента.
6. Очередь-список: Удаление элемента.
7. Очередь-список: Вывод очереди.
8. Очередь-список: Вывод использованных адресов.

9. Очередь-список: Моделирование очереди.
10. Сравнение производительности.
11. Изменение параметров моделирования.

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мною были реализованы следующие структуры данных:

Структура для заявки с временем поступления

<pre>typedef struct { int type; double arrival_time } request_t;</pre>
--

- **type** – тип заявки.
- **arrival_time** – время поступления в систему.

Структура для очереди на массиве

<pre>typedef struct { request_t data[MAX_QUEUE_SIZE]; // 1 000 int pin; int pout; int size; } array_queue;</pre>
--

- **data** – статический массив заявок.
- **pin** – индекс хвоста.
- **pout** – индекс головы.
- **size** – текущий размер.

Структура для элемента списка

<pre>typedef struct node { request_t request; struct node *next; } node_t;</pre>
--

- **request** – заявка.
- **next** – указатель на следующий узел списка.

Структура для очереди на списке

<pre>typedef struct { node_t *pin; node_t *pout; int size; } list_queue;</pre>
--

- **pin** – указатель на узел хвоста.

- **pout** – указатель на узел головы.
- **size** – текущий размер.

Структура для статистики

```
typedef struct
{
    int total_requests_type1;
    int served_requests_type1;
    int total_requests_type2;
    double total_queue_time;
    double total_modeling_time;
    double device_idle_time;
    int max_queue_length;
    double total_queue_length;
    int measurements_count;
} statistics_t;
```

- **total_requests_type1** – всего заявок 1 типа вошло.
- **served_requests_type1** – обслужено заявок 1 типа.
- **total_requests_type2** -- всего обращений заявки 2 типа.
- **total_queue_time** – суммарное время в очереди.
- **total_modeling_time** – общее время моделирования.
- **device_idle_time** – время простоя аппарата.
- **max_queue_length** – максимальная длина очереди.
- **total_queue_length** – сумм длин очередей для усреднения.
- **measurements_count** – количество измерений.

Структура для состояния системы

```
typedef struct
{
    double time;
    double next_arrival_type1;
    double next_service_end;
    int device_busy;
    request_t current_request;
    double service_start_time;
} system_state_t;
```

- **time** – текущее время моделирования.
- **next_arrival_type1** – время следующего прихода заявки 1 типа.
- **next_service_end** -- время окончания обслуживания.
- **device_busy** – занят ли аппарат (0 - свободен, 1 - занят).
- **current_request** – текущая заявка.
- **service_start_time**– время начала обслуживания текущей заявки.

4. Описание Алгоритмов

Основной алгоритм программы:

1. Вывести меню;
2. Считать выбор пользователя и проверить его корректность;
3. Выполнить соответствующее действие;
4. Либо снова вывести меню с ожиданием выбора, либо завершить программу с сообщением об ошибке.

Алгоритм добавления элемента:

1. Для очереди-массива:
 - 1.1. Проверить, не заполнена ли очередь ($size == MAX_QUEUE_SIZE$).
 - 1.2. Записать элемент в позицию pin .
 - 1.3. Увеличить pin с учётом кольцевой структуры: $pin = (pin + 1) \% MAX_QUEUE_SIZE$.
 - 1.4. Увеличить счётчик размера очереди $size$.
2. Для очереди-списка:
 - 2.1. Выделить память под новый узел $node_t$.
 - 2.2. Если память не выделена — вернуть ошибку.
 - 2.3. Записать данные заявки в узел.
 - 2.4. Если очередь пуста:
 - 2.4.1. Установить $rout$ и pin на новый узел.
 - 2.5. Иначе:
 - 2.5.1. Присоединить новый узел к концу очереди ($pin \rightarrow next = \text{новый_узел}$).
 - 2.5.2. Обновить pin на новый узел.
 - 2.6. Увеличить счётчик размера $size$.

Алгоритм удаления элемента:

1. Для очереди-массива:
 - 1.1. Проверить, не пуста ли очередь ($size == 0$).
 - 1.2. Запомнить элемент в позиции $rout$.
 - 1.3. Увеличить $rout$ с учётом кольцевой структуры: $rout = (rout + 1) \% MAX_QUEUE_SIZE$.
 - 1.4. Уменьшить счётчик размера $size$.
 - 1.5. (Опционально) вывести информацию об удалении.
2. Для очереди-списка:
 - 2.1. Проверить, не пуста ли очередь ($rout == NULL$).
 - 2.2. Сохранить ссылку на удаляемый узел ($temp = rout$).

- 2.3. Сохранить данные заявки из узла.
- 2.4. Сдвинуть rout на следующий узел (rout = rout->next).
- 2.5. Если очередь стала пустой (rout == NULL), обнулить pin.
- 2.6. Освободить память узла temp.
- 2.7. Уменьшить счётчик размера size.
- 2.8. (Опционально) вывести адрес удалённого узла.

Алгоритм вставки элемента на позицию:

1. Для очереди-массива:
 - 1.1. Проверить, не заполнена ли очередь.
 - 1.2. Если позиция больше размера — установить позицию = размеру (вставка в конец).
 - 1.3. Сдвинуть элементы от позиции до конца на одну ячейку вправо с учётом кольцевой структуры.
 - 1.4. Вставить новый элемент на свободную позицию.
 - 1.5. Увеличить pin и size.
2. Для очереди-списка:
 - 2.1. Проверить, не заполнена ли очередь.
 - 2.2. Если позиция больше размера — установить позицию = размеру.
 - 2.3. Выделить память под новый узел.
 - 2.4. Если вставка в начало (position == 0):
 - 2.4.1. Новый узел ссылается на rout.
 - 2.4.2. rout становится новым узлом.
 - 2.5. Если вставка в конец (position == size):
 - 2.5.1. pin->next = новый узел.
 - 2.5.2. pin = новый узел.
 - 2.6. Иначе:
 - 2.6.1. Найти узел перед позицией вставки.
 - 2.6.2. Вставить новый узел между найденным и следующим.
 - 2.7. Увеличить size.

Алгоритм моделирования системы массового обслуживания:

1. Инициализация:
 - 1.1. Очистить очередь.
 - 1.2. Установить начальное время `time = 0`.
 - 1.3. Запланировать первое событие — приход заявки 1 типа.
 - 1.4. Поместить заявку 2 типа в обслуживающий аппарат (ОА).
 - 1.5. Обнулить статистику.
2. Основной цикл (пока не обслужено 1000 заявок 1 типа):
 - 2.1. Определить тип следующего события (приход заявки 1 типа или окончание обслуживания).

- 2.2.Обновить статистику длины очереди и времени простоя аппарата.
- 2.3.Перейти ко времени события.
- 3. Обработка события прихода заявки 1 типа:
 - 3.1.Проверить переполнение очереди.
 - 3.2.Создать заявку 1 типа с текущим временем.
 - 3.3.Добавить её в очередь.
 - 3.4.Запланировать следующий приход заявки 1 типа.
- 4. Обработка события окончания обслуживания:
 - 4.1.Если обслужена заявка 1 типа:
 - 4.2.Увеличить счётчик обслуженных заявок.
 - 4.3.Добавить время ожидания в статистику.
 - 4.4.Если обслужена заявка 2 типа:
 - 4.5.Вернуть её в очередь на 4-ю позицию от начала.
 - 4.6.Извлечь следующую заявку из очереди (если есть):
 - 4.7.Определить время обслуживания по типу заявки.
 - 4.8.Запустить обслуживание.
 - 4.9.Иначе:
 - 4.9.1. Освободить аппарат.
- 5. Завершение моделирования:
 - 5.1.Вывести статистику.
 - 5.2.Вернуть результаты.

Алгоритм сравнения производительности:

- 1. Установить количество запусков.
- 2. Оценить используемую память:
 - 2.1.Для массива: `MAX_QUEUE_SIZE * sizeof(request_t)`.
 - 2.2.Для списка: `средняя_длина_очереди * sizeof(node_t)`.
- 3. Измерить время работы для массива:
 - 3.1.Запустить моделирование без вывода ``runs`` раз.
 - 3.2.Замерить время выполнения каждого запуска.
 - 3.3.Вычислить среднее время.
- 4. Измерить время работы для списка:
 - 4.1.Запустить моделирование без вывода ``runs`` раз.
 - 4.2. Замерить время выполнения и среднюю длину очереди.
 - 4.3. Вычислить среднее время и среднюю память.
- 5. Вывести сравнительную таблицу:
 - 5.1.Время работы (мс).
 - 5.2.Используемая память (байт).

Реализуемые функции:

- // Функция очистки буфера ввода
- 1. void clear_input_buffer(void);
- // Функция обновления параметров моделирования
- 2. int update_simulation_parameters(void);
- // Функция вывода сообщения об ошибке
- 3. void print_error(int rc);
- // Функция вывода меню
- 4. void print_menu(void);
- // Функция вывода статистики
- 5. void print_statistics(const statistics_t *stats);
- // Функция вывода промежуточной статистики
- 6. void print_intermediate_stats(const statistics_t *stats, int queue_size, int step);
- // Функция расчета теоретических значений и погрешности
- 7. void calculate_accuracy(const statistics_t *stats);
- // Функция инициализации очереди-списка
- 8. void init_list_queue(list_queue *q);
- // Функция проверки очереди-списка на пустоту
- 9. int is_list_queue_empty(list_queue *q);
- // Функция добавления элемента в очередь-список
- 10. int enqueue_list(list_queue *q, request_t req, int show_address);
- // Функция удаления элемента из очереди-списка
- 11. request_t dequeue_list(list_queue *q, int show_address);
- // Функция вывода очереди-списка
- 12. void print_list_queue(list_queue *q);
- // Функция вывода использованных адресов памяти
- 13. void print_list_addresses(void);
- // Функция вставки элемента на заданную позицию в очередь-список

```
14.int insert_list_element_at_position(list_queue *q, request_t req, int position);

    // Функция инициализации очереди-массива
15.void init_array_queue(array_queue *q);

    // Функция проверки очереди-массива на полноту
16.int is_array_queue_full(array_queue *q);

    // Функция проверки очереди-массива на пустоту
17.int is_array_queue_empty(array_queue *q);

    // Функция добавления элемента в очередь-массив
18.int enqueue_array(array_queue *q, request_t request, int printing);

    // Функция удаления элемента из очереди-массива
19.request_t dequeue_array(array_queue *q, int printing);

    // Функция вывода очереди-массива
20.void print_array_queue(array_queue *q);

    // Функция вставки элемента на заданную позицию в очередь-массив
21.int insert_array_element_at_position(array_queue *q, request_t value, int
    position_from_head);

    // Функция инициализации очереди-списка
22.void init_list_queue(list_queue *q);

    // Функция проверки очереди-списка на пустоту
23.int is_list_queue_empty(list_queue *q);

    // Функция добавления элемента в очередь-список
24.int enqueue_list(list_queue *q, request_t req, int show_address);

    // Функция удаления элемента из очереди-списка
25.request_t dequeue_list(list_queue *q, int show_address);

    // Функция вывода очереди-списка
26.void print_list_queue(list_queue *q);

    // Функция вывода использованных адресов памяти
27.void print_list_addresses(void);
```

// Функция вставки элемента на заданную позицию в очередь-список
28.int insert_list_element_at_position(list_queue *q, request_t req, int position);

// Функция генерации случайного времени в заданном диапазоне
29.double generate_random_time(double min, double max);

// Функция моделирования системы массового обслуживания для очереди-массива
30.int simulate_array_queue(int printing, statistics_t *stats_out);

// Функция моделирования системы массового обслуживания для очереди-списка
31.int simulate_list_queue(int printing, statistics_t *stats_out);

// Функция сравнения производительности двух типов очередей
32.int compare_queues(void);

5. Тесты

Позитивные тесты:

1. Добавление элемента в пустую очередь в виде массива.
2. Добавление элемента в непустую очередь в виде массива.
3. Добавление элемента в пустую очередь в виде списка.
4. Добавление элемента в непустую очередь в виде списка.
5. Удаление из непустой очереди в виде массива.
6. Удаление из непустой очереди в виде списка.
7. Вывод пустой очереди в виде массива.
8. Вывод непустой очереди в виде массива.
9. Вывод пустой очереди в виде списка.
10. Вывод непустой очереди в виде списка.
11. Вывод использованных адресов элементов очереди в виде списка.
12. Моделирование обслуживающего аппарата с использованием очереди в виде массива.
13. Моделирование обслуживающего аппарата с использованием очереди в виде списка.
14. Изменение параметров моделирования.
15. Сравнение эффективности моделирований.

Негативные тесты:

1. Удаление из пустой очереди-массива.
2. Удаление из пустой очереди-списка.
3. Добавление элемента в полную очередь-массив.
4. Добавление элемента в полную очередь-список.
5. Ввод невалидного значения при выборе меню.
6. Ввод невалидного значения при вводе элемента в очередь-массив.
7. Ввод невалидного значения при вводе элемента в очередь-список.

Проверка теоретических расчётов:

Средний интервал прихода заявок 1-го типа: «avg_arrival_interval» = $(0 + 5) / 2 = 2.5$ е.в.

Среднее время обслуживания заявки 1-го типа: «avg_service_time_type1» = $(0 + 4) / 2 = 2.0$ е.в.

Среднее время обслуживания заявки 2-го типа: «avg_service_time_type2» = $(0 + 4) / 2 = 2.0$ е.в.

Для обработки 1000 заявок 1-ого типа требуется $1000 * \text{«avg_service_time_type1»} = 2000$ е.в. Для прихода 1000 заявок 1-ого типа требуется $1000 * \text{«avg_arrival_interval»} = 2500$ е.в. С учётом заявки 2-ого типа, которая обрабатывается в любом случае, система не будет пустой, и время моделирования будет определяться временем обработки. Это подтверждают результаты замеров, так как средняя длина очереди равна 36 элементов. Значит количество обработок заявки 2-ого типа будет примерно равно 1/3 от количества обработок заявок 1-ого типа (так как заявка 2-ого типа после обработки возвращается на 4-ую позицию). $\text{«total_requests_type2»} = 333$.

Значит ожидаемое время моделирования: $\text{«total_requests_type1»} * \text{«avg_service_time_type1»} + \text{«total_requests_type2»} * \text{«avg_service_time_type»} = 1000 * 2 + 333 * 2 = 2666$ е.в.

Проверка на практических данных:

ФИНАЛЬНАЯ СТАТИСТИКА		
Общее время моделирования:	2684.50	е.в.
Время простоя аппарата:	0.00	е.в.
Коэффициент простоя:	0.00	%
Заявки 1 типа:		
- вошедшие:	1072	
- вышедшие:	1000	
Обращения заявки 2 типа: 341		
Длина очереди:		
- максимальная:	75	
- средняя:	36.45	
Среднее время в очереди:	90.29	е.в.
ПРОВЕРКА ТОЧНОСТИ МОДЕЛИ		
Теоретическое время моделирования:	2680.00	е.в.
Фактическое время моделирования:	2684.50	е.в.
Погрешность по времени:	0.17	%
Проверка по ВХОДУ (по поступлению):		
Ожидаемое число заявок:	1073.80	
Фактическое число заявок:	1072	
Погрешность:	0.17	%
Проверка по ВЫХОДУ (по обслуживанию):		
Ожидаемое время обслуживания:	2682.00	е.в.
Фактическое время обслуживания:	2684.50	е.в.
Погрешность:	0.09	%

Погрешность = $100\% * (\text{практическое время моделирования} - \text{теоретическое время моделирования}) / \text{теоретическое время моделирования} = 100 * (2684 - 2680) / 2680 = 0.17\%$

6. Оценка эффективности

Количество запусков – 1000 раз.

Таблица 1. Сравнение полного моделирования

Параметры моделирования	Данные	Массив-очередь	Список-очередь
Приход заявки 1-ого типа: [0;5]	Время (мс)	0.11	0.07
Обслуживание заявки 1-ого типа: [0;4]	Память (байт)	16000	1944
Обслуживание заявки 2-ого типа: [0;4]			
Приход заявки 1-ого типа: [0;10]	Время (мс)	0.09	0.08
Обслуживание заявки 1-ого типа: [0;5]	Память (байт)	16000	120
Обслуживание заявки 2-ого типа: [0;5]			
Приход заявки 1-ого типа: [9;9]	Время (мс)	0.23	0.07
Обслуживание заявки 1-ого типа: [10;10]	Память (байт)	16000	11616
Обслуживание заявки 2-ого типа: [10;10]			
Приход заявки 1-ого типа: [100;100]	Время (мс)	2.37	2.87
Обслуживание заявки 1-ого типа: [1;1]	Память (байт)	16000	24
Обслуживание заявки 2-ого типа: [1;1]			

Таблица 2. Сравнение базовых операций отдельно

Операция	Массив-очередь	Список-очередь
Добавление	0.006 мкс	0.016 мкс
Удаление	0.006 мкс	0.013 мкс

Наблюдение результатов:

1. При полном моделировании список часто оказывается быстрее, особенно когда заявка 2-го типа возвращается в 4-ю позицию, так как массиву приходится сдвигать элементы, а список — только переставлять указатели.
2. При чистых FIFO-операциях (только в конец/из начала) массив работает в 2.2–2.7 раза быстрее: добавление — 0.006 мкс против 0.016 мкс, удаление — 0.006 мкс против 0.013 мкс.
3. Память: список остаётся значительно экономичнее при малой и средней загрузке очереди.
4. Вывод: преимущество той или иной реализации зависит от типа операций:
 - 4.1. Если нужны вставки в середину → список эффективнее.
 - 4.2. Если используется чистый FIFO → массив эффективнее как по скорости, так и по предсказуемости.

7. Выводы

Итак, можно сделать следующие выводы:

Реализация очереди на массиве значительно быстрее при выполнении базовых операций FIFO:

- Добавление — в 2.7 раза быстрее,
- Удаление — в 2.2 раза быстрее.

Это обусловлено отсутствием накладных расходов на динамическое выделение и освобождение памяти, а также лучшей локальностью данных и использованием кольцевого массива.

Однако в условиях реального моделирования, где заявки 2-го типа требуют вставки в середину очереди, преимущество получает список, поскольку массиву приходится выполнять $O(n)$ сдвигов, что замедляет работу.

Таким образом, выбор структуры зависит от характера операций:

- Чистый FIFO → массив (быстрее и проще),
- Модификации в середине → список (гибче и эффективнее).

При этом список всегда выгоднее по памяти, особенно при неполной загрузке очереди.

Фрагментация памяти: при частых удалениях и добавлениях элементов в очередь-список память как переиспользуется, так и оставляет в себе неиспользуемые куски, что свидетельствует о фрагментации памяти.

```
Использованные адреса памяти (18):
0x5f542dd98ac0 0x5f542dd98ac0 0x5f542dd98ac0 0x5f542dd98ae0 0x5f542dd98b00
0x5f542dd98b00 0x5f542dd98ae0 0x5f542dd98ac0 0x5f542dd98b20 0x5f542dd98b40
0x5f542dd98b60 0x5f542dd98b80 0x5f542dd98ba0 0x5f542dd98bc0 0x5f542dd98b20
0x5f542dd98ac0 0x5f542dd98ae0 0x5f542dd98b20
Анализ фрагментации:
Обнаружена фрагментация между 0x5f542dd98ac0 и 0x5f542dd98b20 (разница: 96 байт)
Обнаружена фрагментация между 0x5f542dd98ae0 и 0x5f542dd98b20 (разница: 64 байт)
```

8. Ответы на контрольные вопросы

1. Что такое FIFO и LIFO?

FIFO (First In, First Out) — «первым пришел, первым ушел». Это принцип работы очереди, где элементы обрабатываются в порядке их поступления.

LIFO (Last In, First Out) — «последним пришел, первым ушел». Это принцип работы стека, где последний добавленный элемент извлекается первым.

В данной работе мы моделируем FIFO-очередь, так как заявки обрабатываются в порядке их поступления в систему.

2. Каким образом, и какой объем памяти выделяется под хранение очереди при различной ее реализации?

Реализация массивом:

Память выделяется одним непрерывным блоком фиксированного размера при инициализации.

Объем памяти: $\text{MAX_QUEUE_SIZE} \times \text{sizeof}(\text{request_t}) = 1000 \times 16 = 16000$ байт.

Особенность: память выделяется сразу вся, даже если очередь пуста.

Реализация списком:

Память выделяется динамически под каждый новый элемент (узел) с помощью malloc.

Объем памяти: $n \times \text{sizeof}(\text{node_t}) = n \times 24$ байт (где n — текущее количество элементов в очереди).

Особенность: память выделяется только для фактически хранимых элементов, что экономит ресурсы.

3. Каким образом освобождается память при удалении элемента из очереди при различной реализации?

Реализация массивом:

Процесс: просто перемещается указатель на начало очереди ($\text{pout}++$).

Освобождение памяти: не происходит — память остается зарезервированной под весь массив.

Преимущество: минимальные накладные расходы.

Недостаток: неэффективное использование памяти.

Реализация списком:

Процесс:

- Сохраняется указатель на удаляемый узел.
- Указатель front перемещается на следующий узел.
- Память под удаляемый узел явно освобождается через free().
- Адрес узла добавляется в список освобожденной памяти для отслеживания.

Преимущество: оптимальное использование памяти.

Недостаток: дополнительные затраты на управление памятью и риск утечек.

4. Что происходит с элементами очереди при ее просмотре?

При классическом подходе: просмотр невозможен без извлечения элементов.

В реализованной программе:

- Массив: элементы выводятся от начала к концу очереди без изменения структуры (используется индекс rout и счётчик).
- Список: возможен просмотр всех элементов путем обхода списка от rout до rip. Также можно отображать адреса элементов для анализа фрагментации.

5. От чего зависит эффективность физической реализации очереди?

Эффективность зависит от нескольких факторов:

- Тип операций: Если преобладают операции enqueue и dequeue с конца/начала — обе реализации хороши. Если есть частые вставки/удаления в середину — список выигрывает.
- Размер очереди: При малом размере список экономичнее. При большом и стабильном — массив быстрее.

6. Каковы достоинства и недостатки различных реализаций очереди в зависимости от выполняемых над ней операций?

Массив:

- Быстрый доступ, минимальные накладные расходы, хорошая локальность памяти.
- Фиксированный размер, неэффективное использование памяти при малой заполненности.

Список:

- Динамический размер, экономия памяти при малом количестве элементов.
- Медленнее из-за malloc/free, фрагментация памяти, хуже локальность.

7. Что такое фрагментация памяти, и в какой части ОП она возникает?

Фрагментация — это распределение свободной памяти по множеству мелких несмежных блоков, мешающее выделению крупных участков. Возникает в куче (heap) при частом выделении и освобождении памяти переменного размера (например, при работе со списком).

8. Для чего нужен алгоритм «близнецов»?

Алгоритм «близнецов» (buddy system) предназначен для эффективного выделения и освобождения памяти. Он делит память на блоки размером, равным степени двойки. При освобождении соседние «близнецы» одного размера объединяются, что снижает внешнюю фрагментацию и ускоряет поиск свободного блока.

9. Какие дисциплины выделения памяти вы знаете?

- First Fit — первый подходящий блок.
- Best Fit — наименьший подходящий блок.
- Worst Fit — наибольший свободный блок.
- Next Fit — поиск с места последнего выделения.
- Buddy System — выделение блоков-степеней двойки с объединением «близнецов».

10. На что необходимо обратить внимание при тестировании программы?

- Корректность работы с пустой/полной очередью.
- Обработка ошибок (ERROR_MEM, ERROR_OVERFLOW, ERROR_DELETING).
- Отсутствие утечек памяти (особенно для списка).
- Воспроизводимость результатов (фиксированный seed при замерах).

11. Каким образом физически выделяется и освобождается память при динамических запросах?

При вызове malloc() система находит свободный блок в куче (по одной из дисциплин, например, buddy system) и возвращает его адрес. При free() блок помечается как свободный; если соседний блок тоже свободен — они могут быть объединены (coalescing). Это позволяет повторно использовать память, но требует аккуратного управления, чтобы избежать утечек и повреждений кучи.